# todo-backend

Architecture documentation

October 27, 2026

Revision 4be4fa2

## Overview

todo-backend is a small JSON API for a todo app: users register, log in, and manage their own todo entries. It is written in Rust with **axum** and **tokio**, stores data in **PostgreSQL** through **sqlx**, hashes passwords with **argon2id**, and authenticates requests with **HS256 JWT** bearer tokens. It runs as two containers, the app and its database, via Docker Compose.

The diagrams in this document are generated from the PlantUML sources in docs/architecture/; edit those and rebuild with docs/build.sh.

| # | Diagram | Type | Shows |
|---|---|---|---|
| 1 | System context | use case | Clients, the API and PostgreSQL |
| 2 | Deployment | deployment | Compose services, network, volume, test DB |
| 3 | Components | component | Crate modules and their dependencies |
| 4 | Request pipeline | activity | Tower layers, extractors, error mapping |
| 5 | Register | sequence | POST /auth/register |
| 6 | Login | sequence | POST /auth/login |
| 7 | Update a todo | sequence | PATCH /todos/{id} |
| 8 | Data model | ER | users and todos tables |
| 9 | Models | class | Request and response types, validation |
| 10 | Errors and state | class | AppError, Config, AppState |
| 11 | Lifecycle | activity | Startup, migrations, graceful shutdown |
| 12 | Test layout | component | Test suites and what each exercises |

# Contents

**1 System context** **3**

**2 Deployment** **4**

**3 Components** **5**

**4 Request pipeline** **6**

**5 Registration** **7**

**6 Login** **8**

**7 Authenticated todo request** **9**

**8 Data model** **10**

**9 Request and response models** **11**

**10 Errors, configuration and shared state** **12**

**11 Process lifecycle** **13**

**12 Test layout** **14**

# List of Figures

1 System context . . . 3
2 Deployment (`docker-compose.yml`) . . . 4
3 Crate modules and dependencies . . . 5
4 Request pipeline . . . 6
5 `POST /auth/register` . . . 7
6 `POST /auth/login` . . . 8
7 `PATCH /todos/{id}` . . . 9
8 Database schema . . . 10
9 Request and response models . . . 11
10 Errors, configuration and shared state . . . 12
11 Process lifecycle . . . 13
12 Test suites . . . 14

# 1 System context

The API serves two kinds of client: scripts and tools calling it directly, and browser applications on origins listed in `CORS_ALLOWED_ORIGINS`. All persistent state lives in PostgreSQL, reached through a pool of at most ten connections.

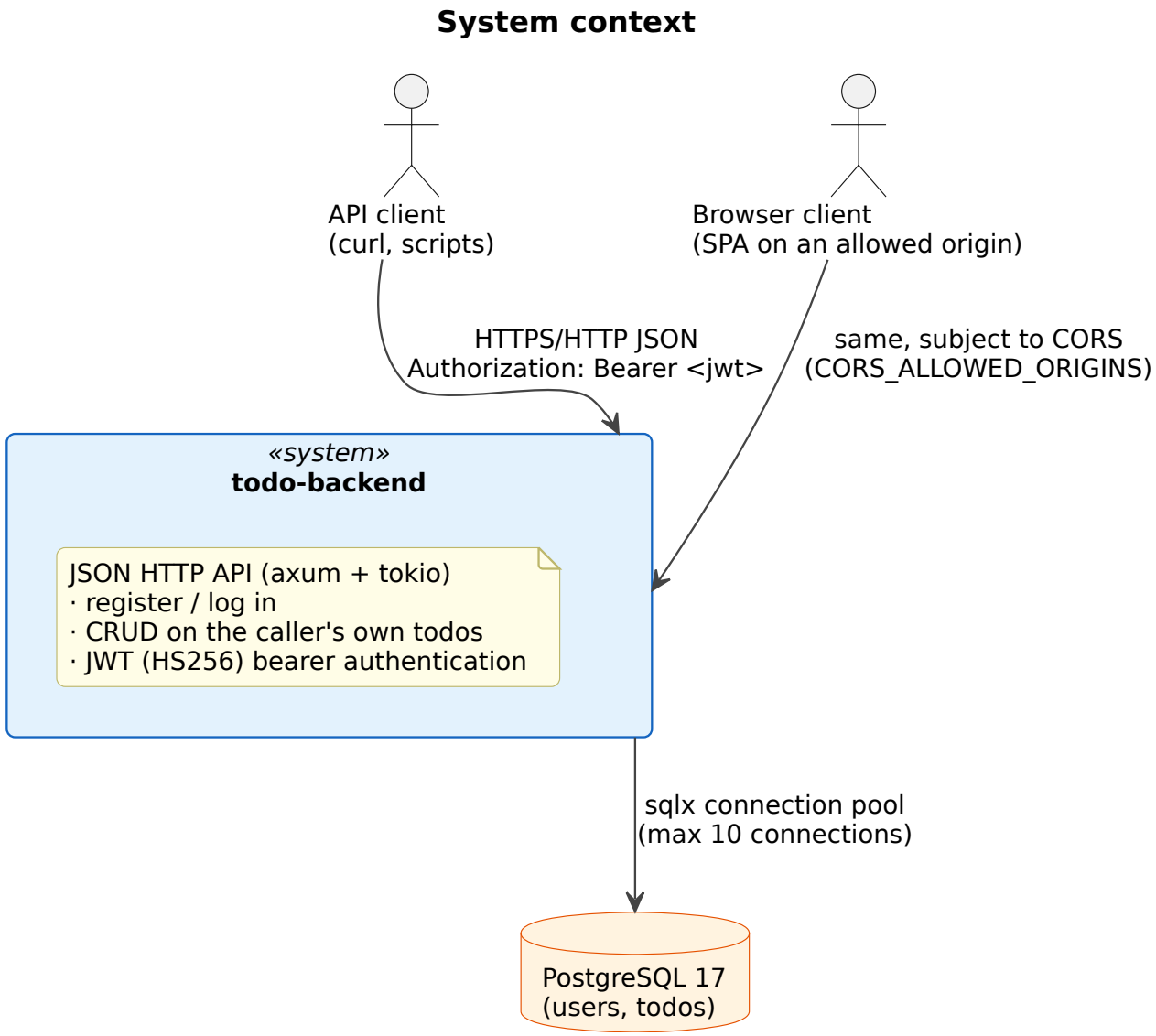



**Figure 1:** System context

## 2 Deployment

Docker Compose runs the app and its database on a private network; only the app's port is published. The `db-test` service, enabled with the `test` profile, is a throwaway database on `tmpfs` for `cargo test` and sqlx's compile-time query checks.

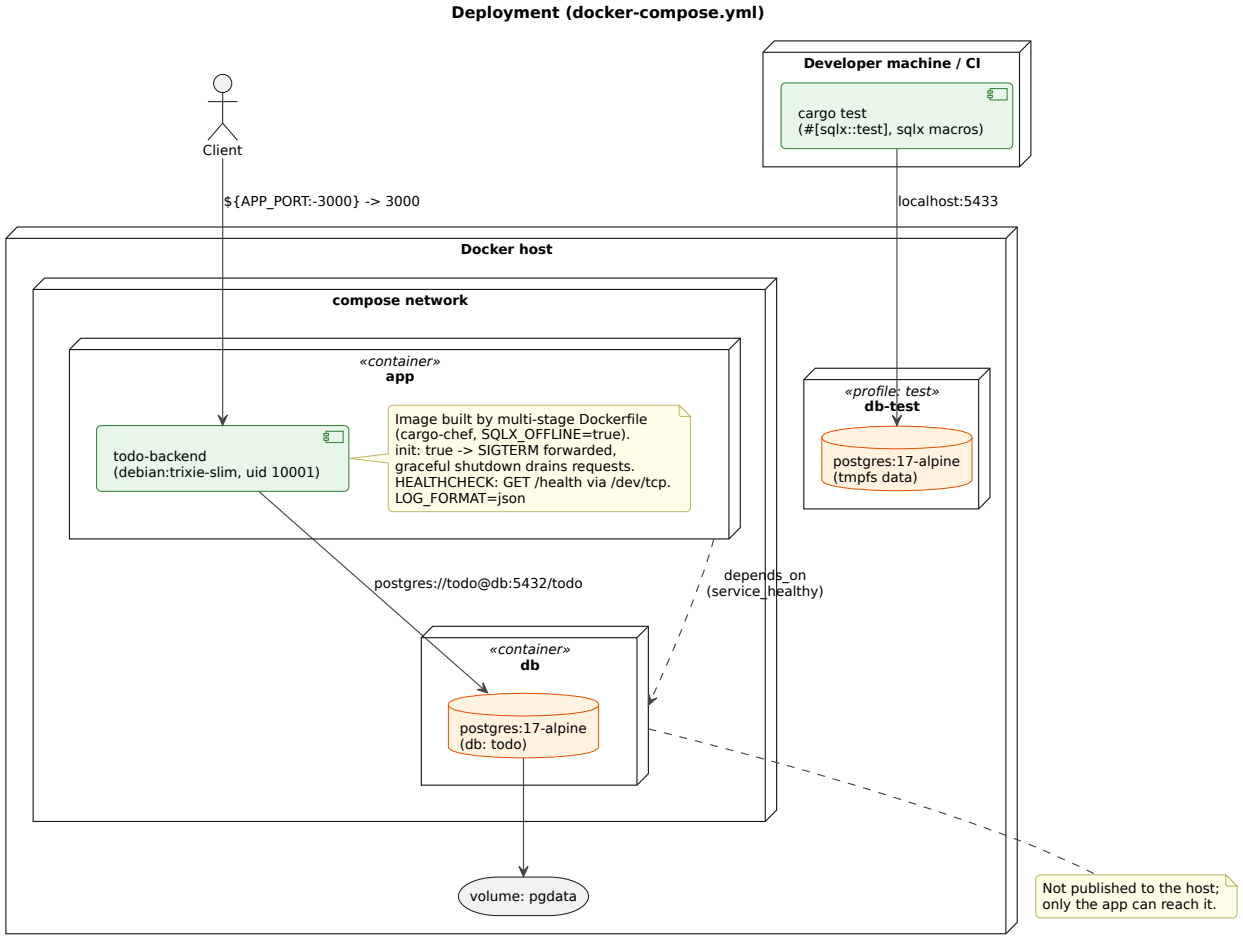



**Figure 2:** Deployment (`docker-compose.yml`)

# 3 Components

`main.rs` loads configuration, initialises tracing, builds the shared `AppState` and hands it to `build_app`. Routes depend on extractors, request models and repositories; only the repositories talk to the database. `AppError` is the single error type shared by every layer.

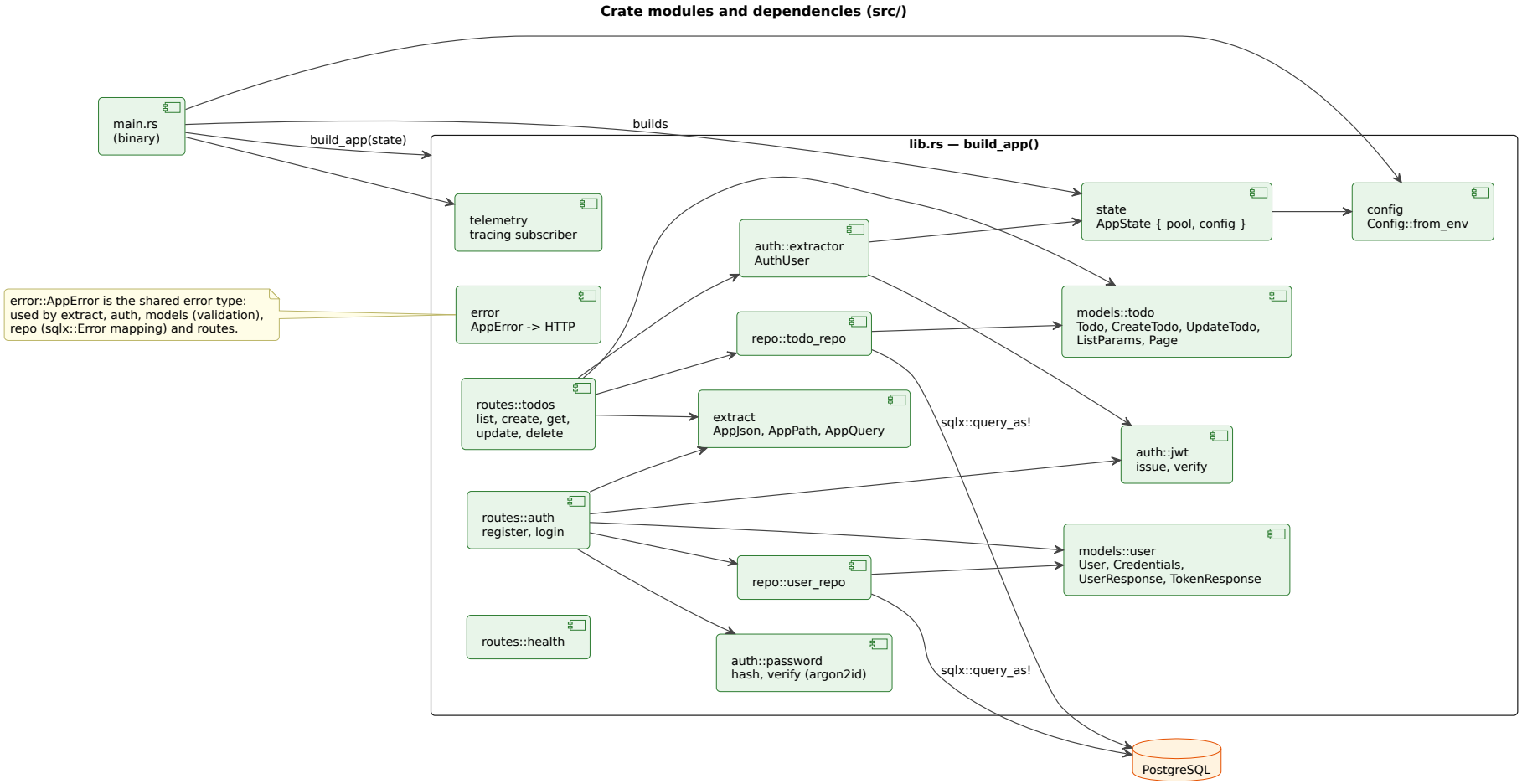



**Figure 3:** Crate modules and dependencies

# 4 Request pipeline

Every request passes through tracing, CORS and the 64 KiB body limit before routing. Handler arguments are custom extractors that turn malformed input into an `AppError`, so every error response has the same `{"error": "..."}` shape.

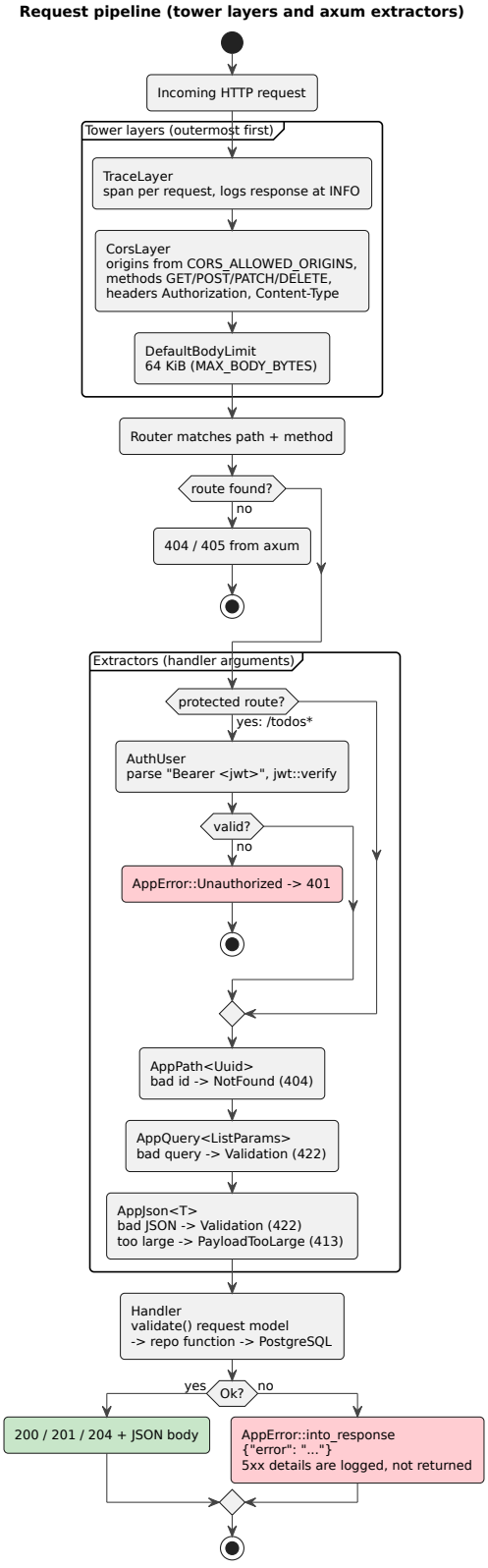



**Figure 4:** Request pipeline

# 5 Registration

Passwords are hashed with argon2id on tokio's blocking thread pool so the CPU-heavy work does not stall the async runtime. A duplicate username surfaces as a unique-constraint violation and is reported as `409 Conflict`.

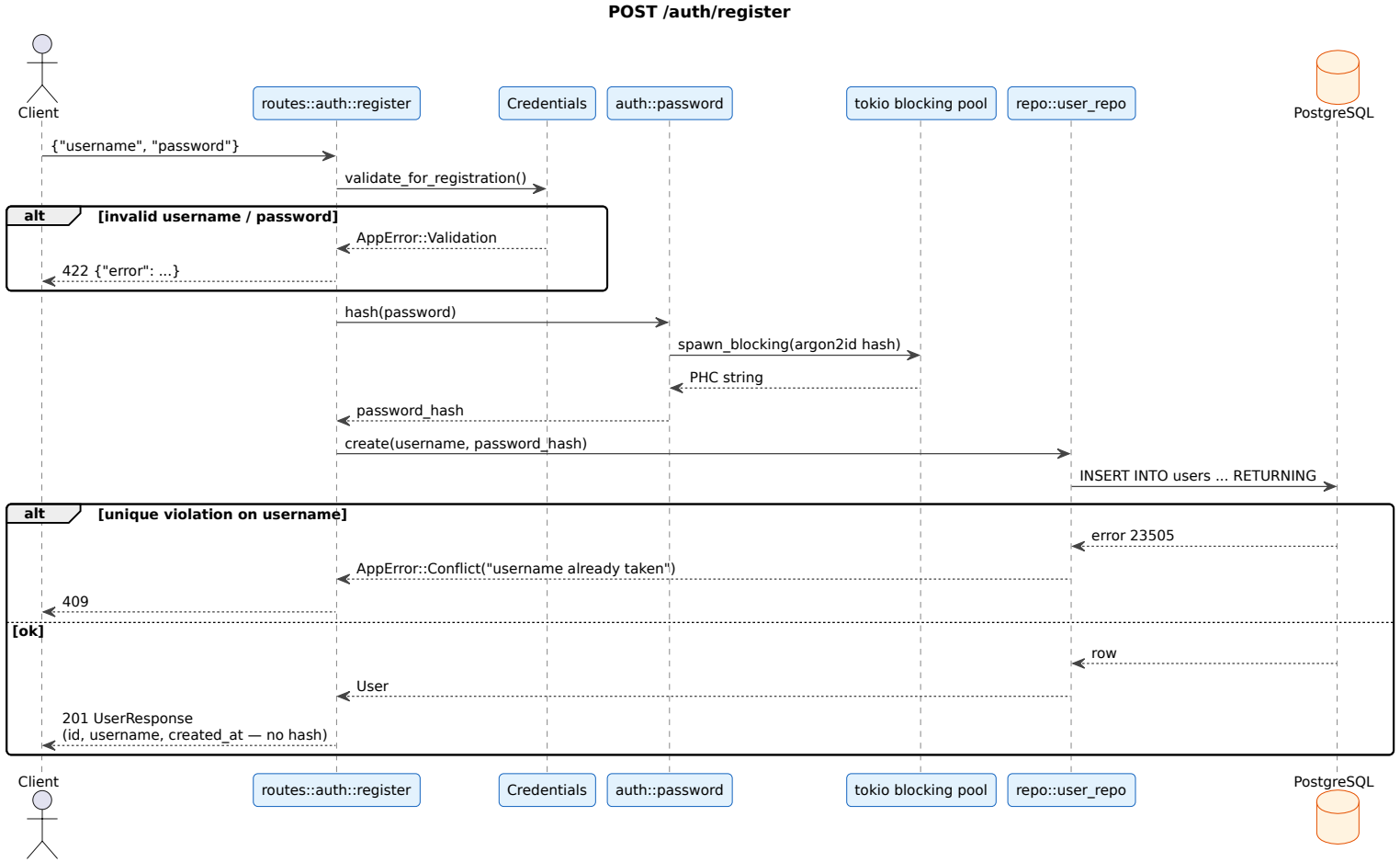



**Figure 5:** `POST /auth/register`

# 6 Login

When the username is unknown, the password is still verified against a dummy hash, so the response time does not reveal which usernames exist. Successful logins receive an HS256 token whose lifetime is `JWT_TTL_SECONDS`.

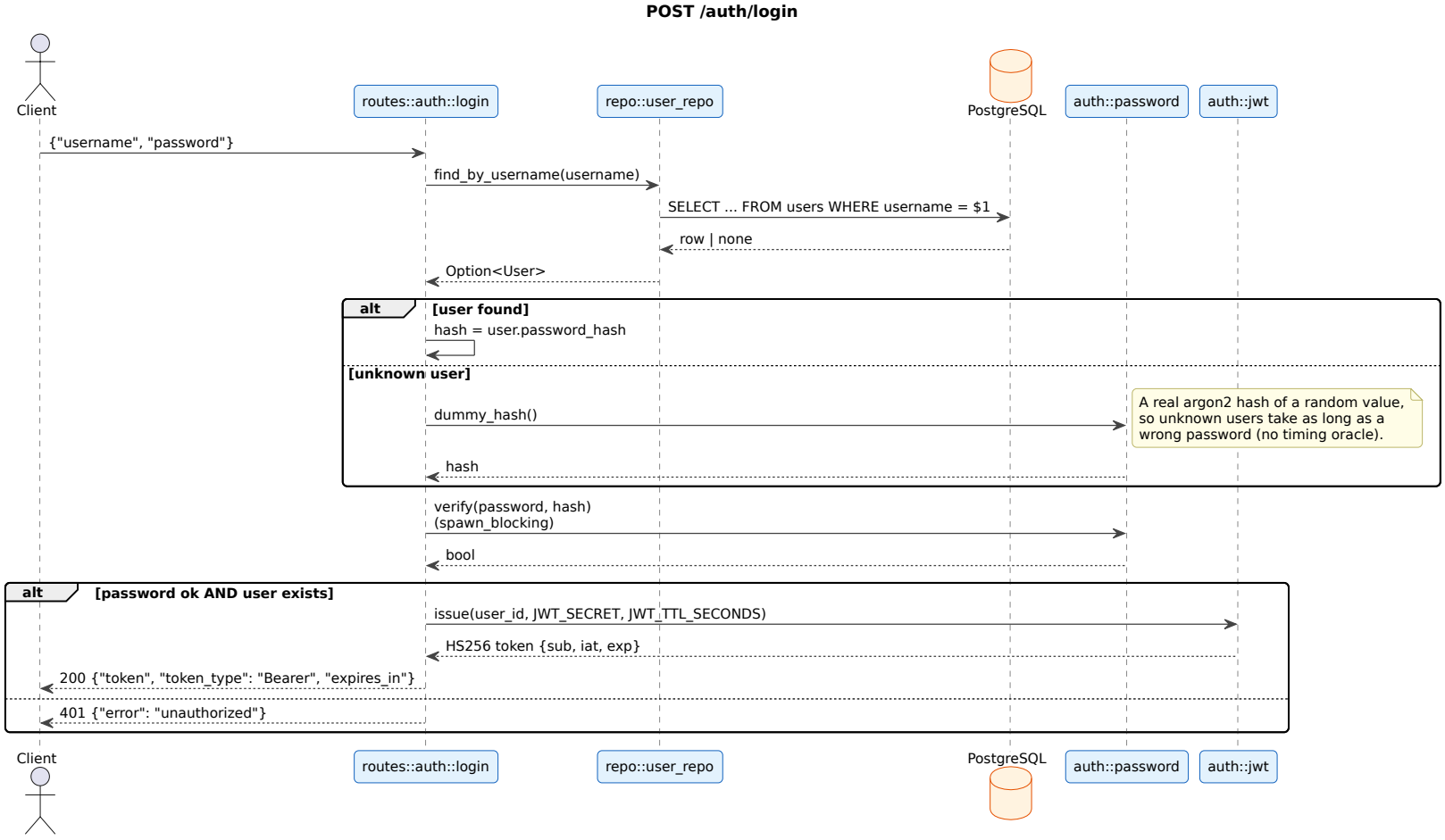



**Figure 6:** `POST /auth/login`

# 7 Authenticated todo request

`PATCH /todos/{id}` is representative of all todo routes: the `AuthUser` extractor verifies the token, the request body is validated into a change set, and the query filters on both the todo id and the caller's id. Another user's todo is therefore indistinguishable from a missing one.

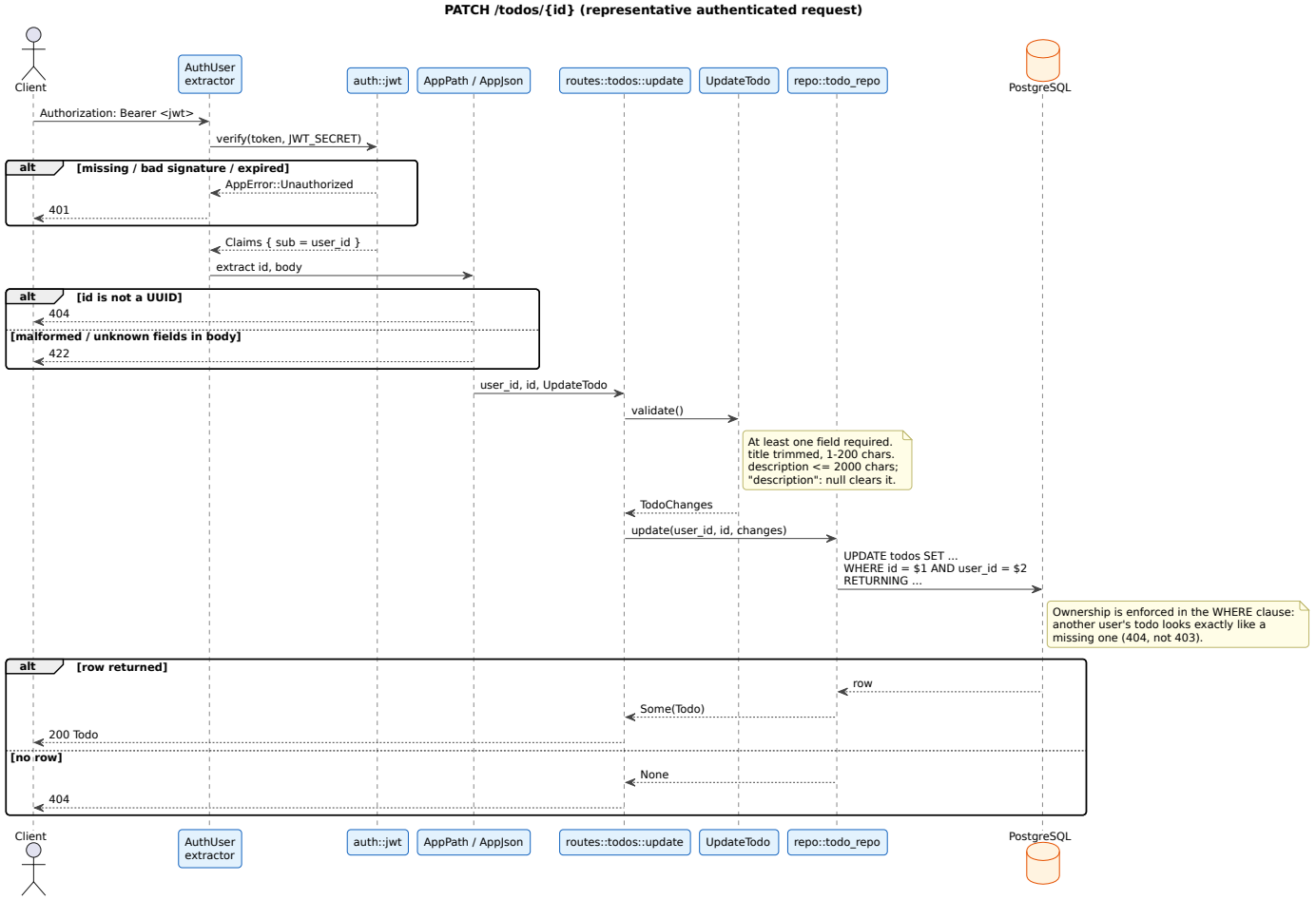



**Figure 7:** `PATCH /todos/{id}`

## 8 Data model

Two tables, created by the migrations in `migrations/` and applied automatically on startup. Deleting a user cascades to their todos.

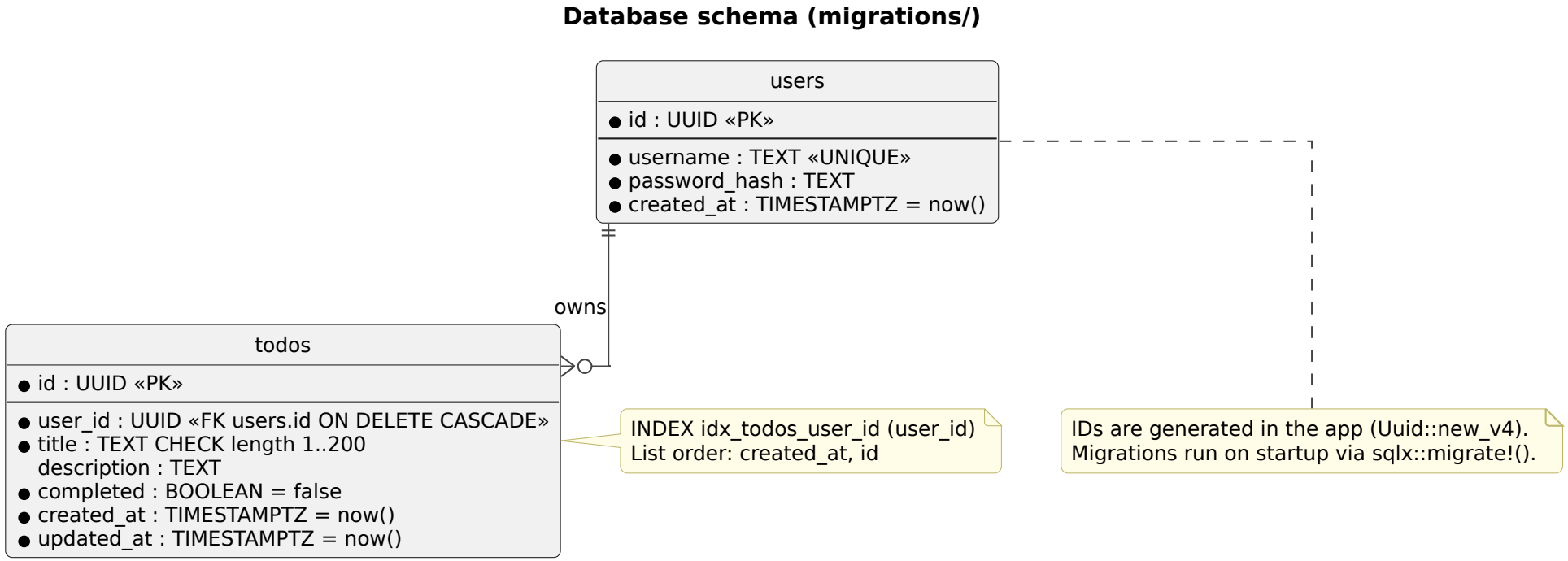



**Figure 8:** Database schema

# 9 Request and response models

Request types are validated into separate, already-checked types before reaching the repositories, so a repository never sees unvalidated input. `User` carries the password hash and is never serialised; responses use `UserResponse`.

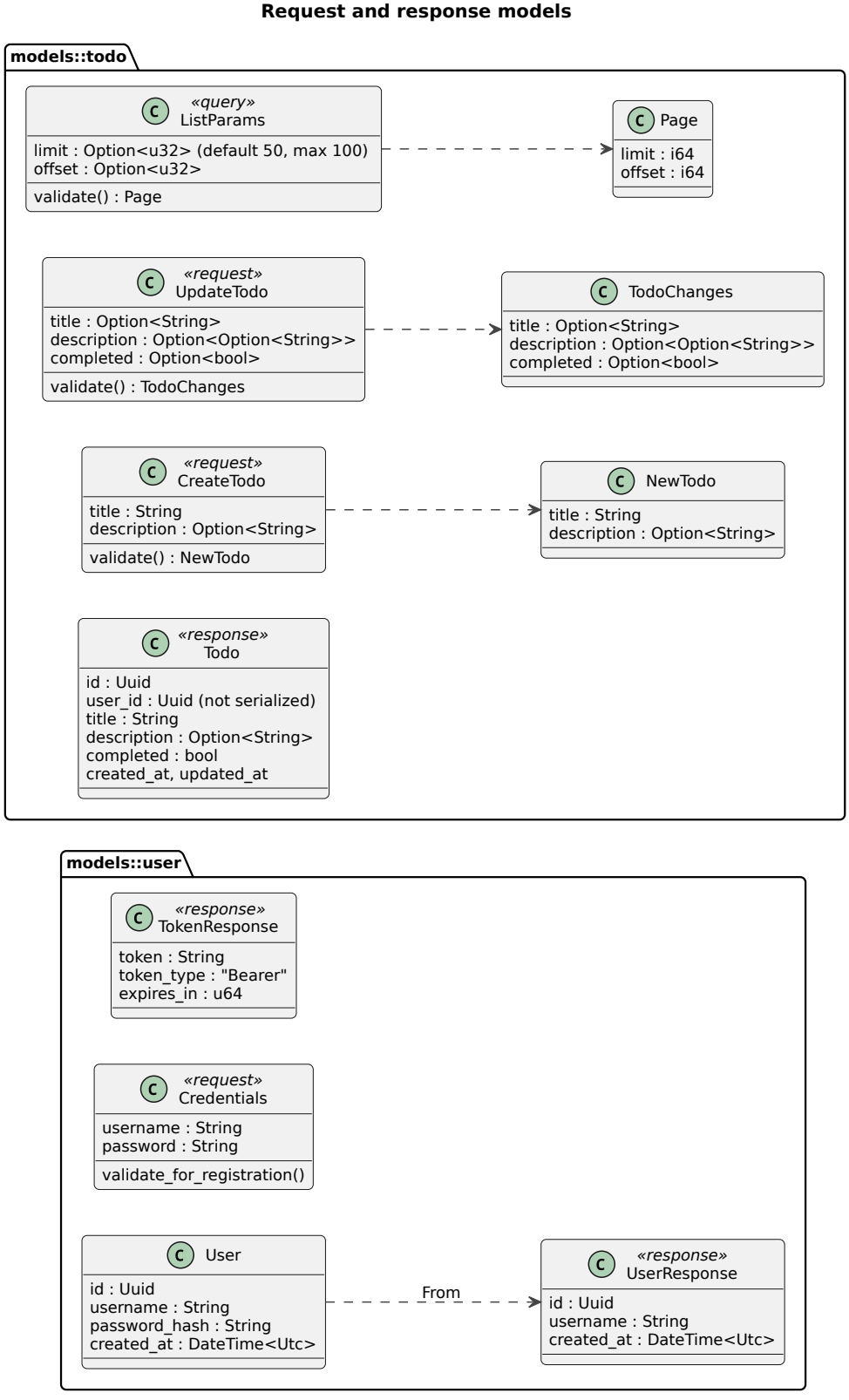



**Figure 9:** Request and response models

## 10 Errors, configuration and shared state

`AppError` maps each failure to an HTTP status; server-side details are logged rather than returned. `AppState` is cloned into every handler and holds the connection pool and the configuration loaded at startup.

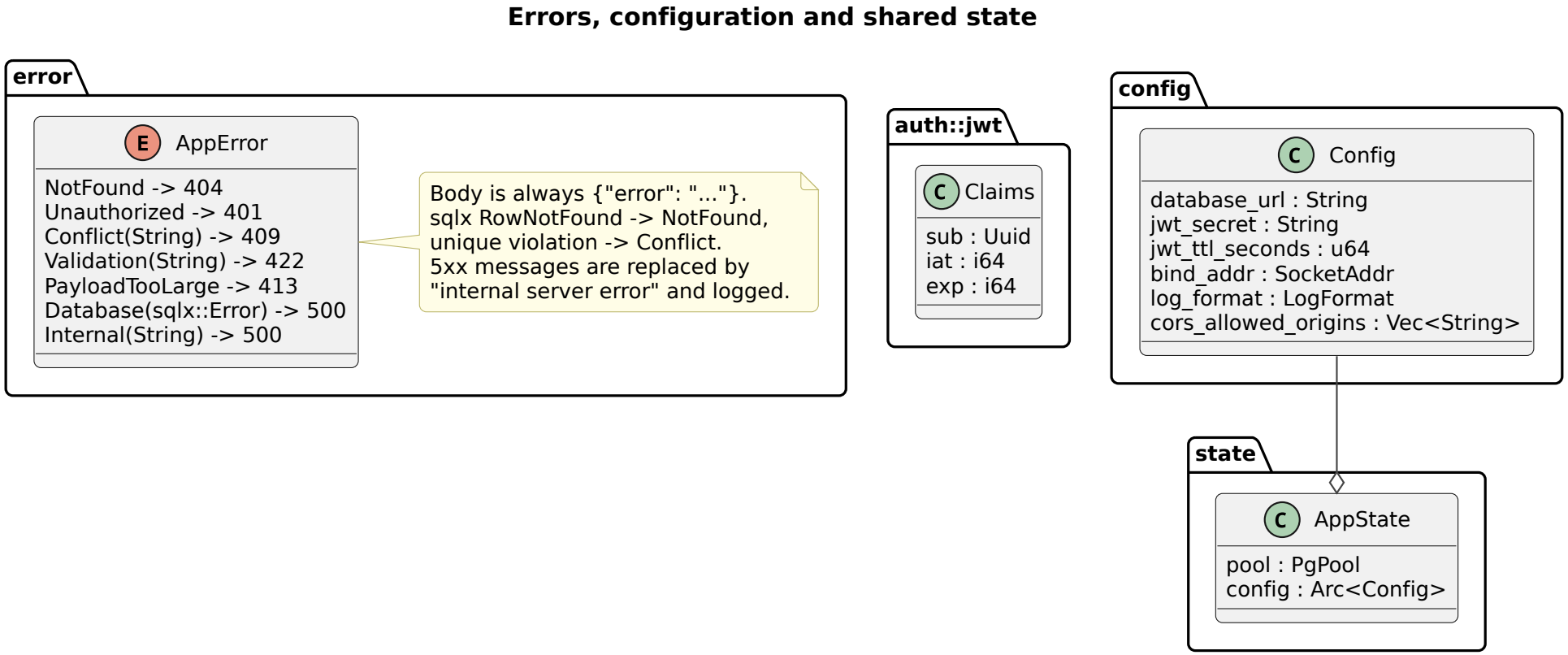



**Figure 10:** Errors, configuration and shared state

# 11 Process lifecycle

Configuration errors abort startup with a message. On SIGTERM (as sent by `docker stop`) or Ctrl-C the server stops accepting connections, lets in-flight requests finish, and closes the pool.

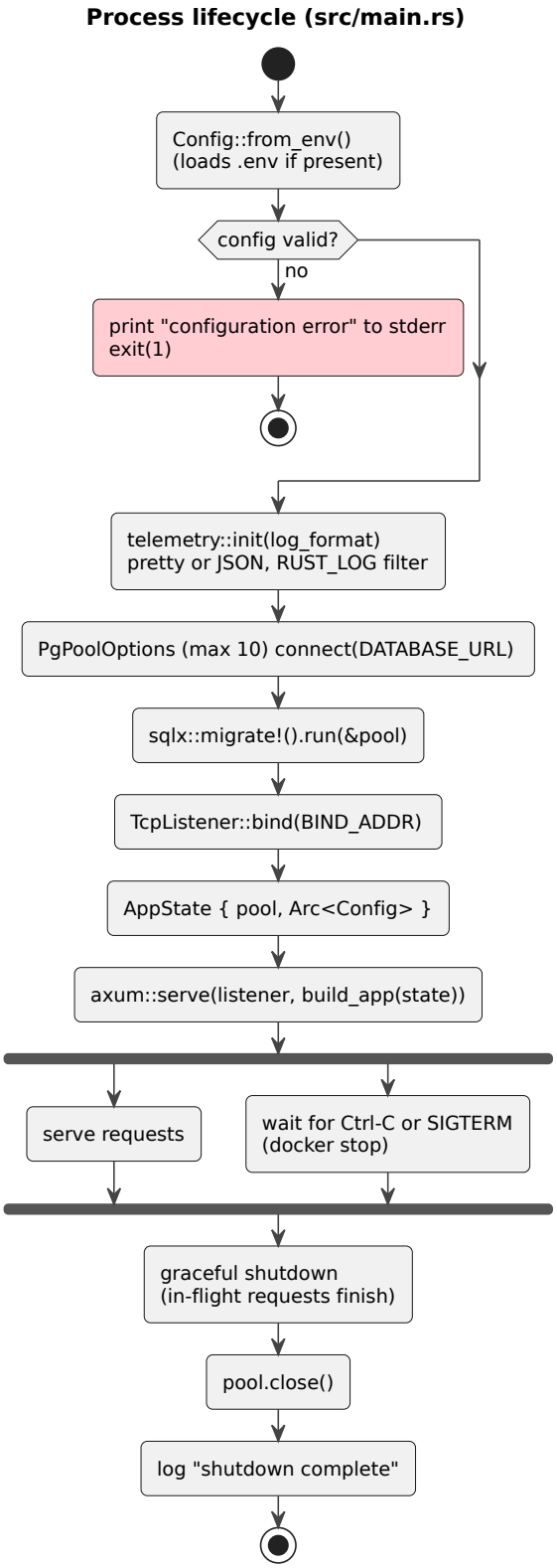



**Figure 11:** Process lifecycle

## 12 Test layout

Unit tests cover pure logic. Repository and API tests run against a fresh database per test via `#[sqlx::test]`, with the API exercised in-process through `tower`. End-to-end tests drive the real Docker Compose stack over HTTP and run in CI.

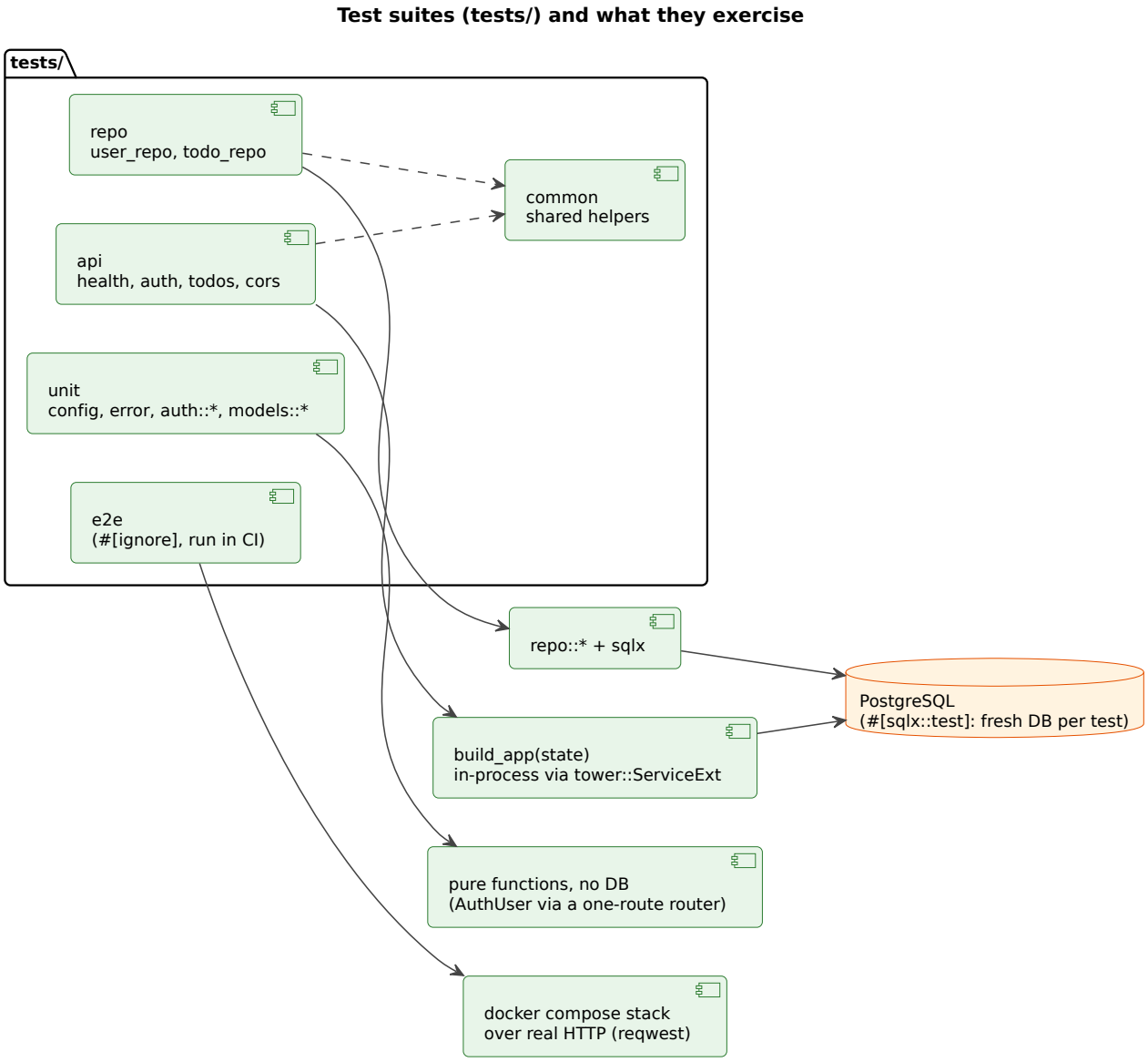



**Figure 12:** Test suites